

System Maintenance Document

Team Members: Sanidhya Verma (Representative),

Md Tariqul Alam Khandaker, Manmeet Singh Karwal, Md Faisal Ahmed, Yosef Mohammed H
Alshammari, Sai Prakash Ravichandran

Copyright and Warranty Information

This document and the information it contains are the intellectual property of the HELP Web App project team GQ (Good Question) from La Trobe University. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the authors.

All trademarks and product names mentioned are the property of their respective owners.

This document is provided “as is” without warranty of any kind, either expressed or implied, including but not limited to the implied warranties of merchantability and fitness for a particular purpose. The authors shall not be held liable for any errors or omissions or for any damages resulting from the use of this documentation or the system it describes.

This document and the HELP Web App system are created solely for educational purposes as part of the university curriculum. The software is not intended for production deployment without further review and testing.

Contents

1. Introduction	4
1.1 System Overview:	4
1.2 Audience Description:	4
1.3 Applicability Statement:	4
1.4 Purpose Statement	5
1.5 Document Usage Description	5
1.6 Conventions	6
1.7 Changelog	6
2. Software Design Scope	7
2.1 Major Features:	7
2.2 Design Constraints	7
2.3 Limitations.....	8
2.4 Technology Stack and Design Justification	8
2.5 Design Discussion	9
2.6 Limitations and Future Improvements	10
3. Reference and Vendor Documentation	13
3.1 Reference Documents	13
3.2 Existing System Documentation.....	13
3.3 Vendor and Tool Documentation.....	13
3.4 Minimum System Requirements	14
3.5 Dependency & License Table	15
4. Installation & Maintenance Instructions	17
4.1 System Requirements.....	17
4.2 Installation Instructions	17
4.3 Updating & Patching	18
4.4 Dependency, CVE & License References	18
4.5 Extending the System	18
4.6 API Keys & Environment Configuration	19
4.7 Resources:.....	19
4.7.1 Learning Resources	19
4.7.2 Domain Knowledge Resources	19
5. User Stories and Testing	21
5.1 User Story Definitions	21
5.2 User Story Dictionary.....	21
5.3 User Story Testing Table	23

5.4 Functional Testing Summary.....	24
6. Software Architecture & Diagrams.....	25
6.1 High-Level Architecture Diagram	25
6.2 Sequence Diagram	26
6.3 Use Case Diagram	26
6.3.1 Cyber Threat Model.....	29
6.4 Wireframes	30
6.5 Class Diagram	33
6.6 Entity-Relationship (ER) Diagram	36
7. Database Design	39
7.1 Database Overview	39
7.2 Entity-Relationship Structure.....	39
7.3 Key Collections and Schemas	39
7.4 Additional Notes	41
8: Usability Testing	42
8.1 Objectives of Testing Scenarios	42
8.2 Participants	42
8.3 Methods	42
8.4 Results of Usability Testing	42
8.5 Improvements After Testing	43
9. Project Management Summary	44
9.1 Project Tracking & Sprints	44
9.2 Team Contributions Overview	44
9.3 GitHub Version Control.....	45
9.4 Team Communication	45
9.5 Evaluation & Lessons Learned	46
10. Glossary, Indexes, and Final Note.....	47
10.1 Glossary of Terms	47
10.2 Indexes	47
10.3 Final Notes	48
10.5 Appendix.....	48
10.5.1 Appendix A.....	48
10.5.2 Appendix B.....	50
10.5.3 Appendix C	52

1. Introduction

1.1 System Overview:

The Healthy End of Life Planning (HELP) Web App is a full-stack, accessible platform designed to assess and improve individuals' **death literacy** through a personalized quiz experience. Built using the modern **MERN stack (MongoDB, Express, React, Node.js)**, it features:

- A **responsive, public-facing quiz** with conditional logic and Likert-style responses
- A **secure admin portal** to manage quiz content, view submissions, analyze SEAK domain scores (Skills, Experience, Action, Knowledge), and export data
- **User role management** supporting Public Users, Authenticated Users, and Admins/Researchers with protected route access
- **Accessibility considerations** for older adults, including large fonts, intuitive navigation, and mobile responsiveness
- **Optional AI-powered feedback** integration (using OpenAI) for enhanced personalization and insights

The system is designed for future extensibility and supports research, policy planning, and end-of-life literacy campaigns.

1.2 Audience Description:

This document is intended for the following audiences:

- **System Administrators & Developers:** To maintain, update, and extend the system.
- **Future Student Teams or Developers:** For understanding the design, setup, and extension methodology.
- **Project Stakeholders or Assessors:** To evaluate the system's maintainability and technical architecture.

Readers should be familiar with basic web development tools (e.g., Node.js, MongoDB, React), GitHub version control, and basic Linux command line usage.

1.3 Applicability Statement:

The HELP Web App runs on all modern desktop and mobile web browsers. The development and testing were conducted on:

- Chrome (v122+) and Firefox (v123+)
- Node.js v18+
- MongoDB Community Edition (v6+ or MongoDB Atlas)
- React 18 with Vite for frontend bundling
- Express.js (v4.18+) for backend routing

The app is deployable on local environments or cloud platforms like **Render**, **Vercel**, or **Heroku** (for the frontend) and **MongoDB Atlas** (for database).

1.4 Purpose Statement

This document serves as a comprehensive maintenance reference for the HELP Web App project. It aims to:

- Provide setup and installation instructions for development and production.
- Document all dependencies, license types, and known security vulnerabilities.
- Guide developers in updating, patching, or extending the codebase.
- Act as a technical handover to future teams or contributors.

1.5 Document Usage Description

The document is organized into ten main sections to provide a comprehensive guide to the HELP Web App, covering everything from design to maintenance and project tracking:

- **Section 1: Introduction**
Presents an overview of the system, intended audience, document purpose, and changelog.
- **Section 2: Software Design Scope**
Details the system's major features, design constraints, technology stack, and architectural discussions including scalability, flexibility, and future improvements.
- **Section 3: Reference and Vendor Documentation**
Includes references to existing documentation, vendor tools, system requirements, and dependency/license tables.
- **Section 4: Installation & Maintenance Instructions**
Describes how to install, update, and extend the system, along with patching procedures and CVE/license documentation.
- **Section 5: User Stories and Testing**
Provides a detailed list of user stories, testing plans, and testing outcomes related to core system functionality.

- **Section 6: Software Architecture & Diagrams**
Includes visual diagrams such as high-level architecture, sequence diagrams, use case diagrams, class and ER diagrams, as well as wireframes and a cyber threat model.
- **Section 7: Database Design**
Explains the database schema, key collections, and relationships, with additional design notes for extensibility.
- **Section 8: Usability Testing**
Summarizes the testing objectives, methodology, results, and post-testing improvements made to the system.
- **Section 9: Project Management Summary**
Describes team processes including sprint tracking (Jira), GitHub version control, team communication, contributions, and reflections.
- **Section 10: Glossary, Indexes, and Final Notes**
Provides definitions of key terms, document indexing, and final remarks, followed by appendices including domain resources, learning materials, and extended documentation.

1.6 Conventions

- File paths use Unix-style slashes (/) for consistency.
- Code snippets use monospace formatting.
- Questions in the quiz are referred to by Q[number] or Q[number]_[sub].
- All diagrams follow standard UML or block diagram styles, created using draw.io or Lucidchart.
- The term “admin” refers to both Admin and Researcher roles unless otherwise specified.

1.7 Changelog

Version	Date	Author	Description
1.0	June 2025	Good Question Team	Initial release.

2. Software Design Scope

2.1 Major Features:

The HELP Web App consists of the following major features:

- **Public Quiz Interface**
A 25-question quiz with conditional logic, grouped questions, Likert, text and multiple-choice support, progress bar, question tracker, and review/edit functionality.
- **Score & Feedback System**
Scoring based on selected questions (Q16–Q21), converted to a 0–10 scale per SEAK domain. Feedback is mapped dynamically via scoreInterpretations to compare with benchmark score.
- **User Authentication & Resume**
Users can sign up, log in, and resume quiz from where they left off using localStorage and user-based tracking.
- **Admin Portal**
Allows admins and researchers to:
 - o View quiz submissions
 - o Filter and export results
 - o View detailed submission report
 - o Add/edit/delete quiz questions, including grouped and conditional ones (Admin Only)
 - o Add/edit/delete admin/researcher accounts (Admin Only)
 - o Add/edit/delete benchmark score to compare individual results. (Admin Only)
 - o Access analytics (charts, maps, domain stats)
 - o Download individual submissions
- **Accessibility & Responsiveness**
UI is optimized for all screen sizes and designed to be usable by older adults.

2.2 Design Constraints

- Built using **React + Tailwind** for fast, responsive frontend
- Uses **Node.js + Express + MongoDB** backend
- All quiz logic is dynamic and question-based (no hardcoded logic)
- Local development only (not deployed yet)
- Follows accessibility standards (e.g., large tap targets, high contrast)

2.3 Limitations

- Not deployed publicly (local use or test server only)
- AI feedback system is under development (Needs Payment and verification)
- No formal user login history or audit log yet
- Analytics features are basic (bar/line charts only)

2.4 Technology Stack and Design Justification

Component	Technology	Justification
Frontend	React + Tailwind CSS (via Vite)	Modern SPA with reusable components, fast loading, and responsive styling
Backend	Node.js + Express	Lightweight and scalable, ideal for handling RESTful API requests and logic
Authentication	Google OAuth2 + JWT	Industry-standard login with secure, stateless token management
Database	MongoDB Atlas	NoSQL document-based model suits flexible quiz structure and dynamic responses
AI Integration	OpenAI GPT API	Enables personalized feedback based on user score profile
Hosting (Planned)	Vercel (Frontend), Railway (Backend)	CI/CD-friendly, scalable, and cost-effective for deployment (pending client decision)
Development Tools	VSCode, GitHub, draw.io	Efficient development and version control; diagramming for architecture visuals

Summary

The technology stack for the HELP Web App was chosen to ensure a balance between **developer productivity**, **system scalability**, and **user accessibility**.

- The **frontend**, built with **React + Tailwind CSS**, ensures a fast, accessible, and responsive user experience optimized for all devices.
- The **backend** uses **Node.js + Express**, offering high performance for handling quiz logic, scoring, and API communication.
- **MongoDB Atlas** provides a document-based NoSQL structure, ideal for storing diverse response types, dynamic question sets, and flexible scoring models.

- **Authentication** is handled via **Google OAuth2**, combined with **JWTs** for protected routes and role-based access.
- The optional **OpenAI GPT integration** allows for intelligent, domain-based feedback.
- While currently hosted locally via GitHub, the platform is ready for deployment via **Vercel** (frontend) and **Railway** (backend), pending client direction.
- **Development and planning tools** like GitHub and draw.io were used to manage code, sprints, and architecture diagrams effectively.

2.5 Design Discussion

Scalability

The HELP Web App is designed with **horizontal scalability** and future growth in mind. The backend operates as a **stateless API** built using Node.js and Express, allowing additional instances to be added behind a load balancer. While deployment is currently local and hosted via GitHub, the system is **deployment-ready** for scalable cloud platforms such as **Vercel** (frontend) and **Railway/Render** (backend). The use of **MongoDB Atlas** supports scalability through **automatic sharding, high availability clusters**, and flexible schema design — ensuring the database layer can accommodate increasing volumes of quiz content and user submissions.

Flexibility

Flexibility is achieved through a **modular, decoupled architecture**. The frontend is built in **React**, allowing for **component reusability** and easy updates to the UI without affecting the full system. Backend logic follows **RESTful principles** and separates quiz handling, scoring, and user management into clear modules. This structure allows developers to add new quiz types, analytics tools, or role-based features **without refactoring the core codebase**, making long-term maintenance and expansion efficient.

Connectability

The system is built for seamless integration with third-party tools and services:

- **Google OAuth2** is used for secure, external authentication
- The backend exposes **RESTful APIs** that can be consumed by other applications (e.g., mobile apps, external dashboards)
- The backend includes an integration with the **OpenAI GPT API** for personalized feedback generation, demonstrating how plug-and-play enhancements can be embedded without major system redesign

The architecture allows easy onboarding of **future integrations** like CRM systems, research tools, or health service platforms.

Usability

The platform is designed with **accessibility and simplicity** in mind, particularly for older adults or less tech-savvy users. The **Tailwind CSS**-based frontend is fully **responsive**, with large buttons, readable fonts, and consistent navigation. Features like:

- Quiz pausing/resuming
- Progress indicators
- Review/edit before submission
- Downloadable result reports

... all contribute to a **user-centered experience**. Additionally, **role-based access control** ensures that each user type (Public, Authenticated, Admin, Researcher) only sees the functionality relevant to them — reducing clutter and confusion.

Security

Security is enforced through multiple layers:

- **Google OAuth2** handles authentication, minimizing risks from password storage and credential theft.
- All **protected routes** in the backend are guarded with **middleware-based role checks**.
- **MongoDB Atlas** ensures encryption at rest and in transit.
- Personally identifiable data is minimized, and **passwords are never stored in plaintext**.
- The **frontend does not directly communicate with the database**; all interactions occur via secure HTTPS API endpoints.

This approach ensures data privacy, system integrity, and reduced vulnerability exposure.

2.6 Limitations and Future Improvements

Current Limitations

Despite being functional, modular, and scalable, the current version of the HELP Web App has certain limitations, primarily due to **time constraints**, **resource availability**, and **free-tier infrastructure**.

1) Free-tier Hosting Constraints

- The application currently relies on **GitHub-hosted code**, with optional deployment planned via **Vercel (frontend)** and **Railway (backend)**.
- These platforms, in their free tiers, may introduce:
 - o Cold starts
 - o Rate-limiting
 - o Limited compute time and bandwidth
- Under high user load, these could cause performance or availability issues in a production setting.

2) Third-Party Dependencies

- a. The system integrates **Google OAuth2** for authentication and **OpenAI's GPT API** for AI-generated feedback.
- b. While these integrations enhance functionality, they introduce:
 - i. API dependency risk (downtime, breaking changes, pricing changes)
 - ii. Privacy and data transmission concerns (managed via secure tokens)
- c. Ongoing monitoring and fallback mechanisms will be required for long-term stability.

3) Limited Accessibility Features

- The platform currently supports only **English**, limiting usability for **non-English-speaking** audiences.
- There is **no offline mode**; users must maintain an active internet connection to complete the quiz and receive feedback.
- Although accessibility was a design priority (e.g., large fonts, mobile responsiveness), **screen reader support and WCAG compliance testing** are still pending.

Planned Enhancements

To strengthen inclusivity, usability, and analytical capabilities, the following improvements are under consideration:

1) User Engagement & Feedback:

- a. Introduce **email notifications** for:
 - Quiz result summaries
 - Custom domain insights
 - Updates from researchers/admins

2) Admin Panel Expansion:

a. Add **advanced analytics**:

- Interactive charts (via Chart.js or D3.js)
 - Heatmaps or radar plots for SEAK domain trends
 - User activity and engagement metrics
- b. Implement **content moderation tools** for quiz updates or flagged submissions
- c. Improve filtering in Submissions and Analytics tabs

3) Accessibility & Reach:

- a. Add **multilingual support**, beginning with commonly spoken Australian community languages
- b. Implement **offline quiz mode** with local caching and sync-on-connect functionality
- c. Consider **progressive web app (PWA)** conversion for installable experience on mobile

4) System Resilience:

- a. Add **fallback behaviour** for OpenAI API calls (e.g., default feedback messages)
- b. Monitor for **OAuth or API deprecations** using automated alerts

3. Reference and Vendor Documentation

3.1 Reference Documents

The following documents were used to guide the development and maintenance of the HELP Web App:

- Internal team documentation and wireframes
- Client briefing notes and clarification emails
- Domain-specific materials on Death Literacy and SEAK scoring
- Technical specifications from the university
- Open-source documentation from React, Express, MongoDB, and TailwindCSS
- Accessibility guidelines (WCAG 2.1)

3.2 Existing System Documentation

- **README.md** in the GitHub repository outlines:
 - Project setup and dependencies
 - Available scripts (frontend and backend)
 - Environment variable configuration
- **Postman Collection** (optional): for backend API testing
- **Swagger/Route list** (if applicable): internal notes on REST API endpoints
- **Quiz Content JSON**: defines dynamic quiz flow with sub-questions and categories

3.3 Vendor and Tool Documentation

Frontend Technologies

- **React**
A JavaScript library for building user interfaces
<https://reactjs.org/docs/getting-started.html>
- **Tailwind CSS**
A utility-first CSS framework for rapid UI design
<https://tailwindcss.com/docs>
- **Vite**
A lightning-fast build tool and dev server for modern web projects
<https://vitejs.dev/guide/>

Backend Technologies

- **Node.js**
JavaScript runtime environment
<https://nodejs.org/en/docs>
- **Express.js**
Minimal and flexible web application framework for Node.js
<https://expressjs.com/en/starter/installing.html>
- **MongoDB**
NoSQL document database
<https://www.mongodb.com/docs/manual/>

Authentication & Security

- **JWT (jsonwebtoken)**
Library for secure user authentication
<https://github.com/auth0/node-jsonwebtoken>
- **bcrypt**
For password hashing and comparison
<https://github.com/kelektiv/node.bcrypt.js>

Admin UI & Utilities

- **react-router-dom**
For routing and protected pages
<https://reactrouter.com/en/main>
- **framer-motion**
Animation library used in transitions and quiz UI
<https://www.framer.com/motion/>
- **react-csv**
Utility for exporting submissions to CSV format
<https://www.npmjs.com/package/react-csv>
- **Chart.js / react-chartjs-2**
Charting library for domain analytics
<https://www.chartjs.org/docs/latest/>

3.4 Minimum System Requirements

To ensure smooth usage of the HELP Web App platform across user roles (participants, admins, and researchers), the following minimum system requirements are recommended:

Component	Minimum Requirement
Operating System	Windows 10+, macOS 11+, or Ubuntu 20.04+
Processor	Dual-Core 2.0 GHz or higher

Component	Minimum Requirement
Memory (RAM)	4 GB (8 GB recommended for admin panel usage and data export)
Web Browser	Chrome v95+, Firefox v90+, Safari v15+, or Edge v95+
Display	1024×768 resolution minimum (responsive design supports mobile)
Internet Access	Stable broadband (2 Mbps or higher); required for login & AI features

These specifications are based on the environments used during development and QA testing. The application relies on modern JavaScript features and external APIs (e.g., Google OAuth and OpenAI GPT), which require up-to-date browsers and consistent internet access. For best results, desktop use is recommended—especially for admins and researchers managing analytics or large data sets.

3.5 Dependency & License Table

The following table lists the major software dependencies used in the HELP Web App along with their licenses, purpose, and known CVE considerations. All open-source libraries are used in compliance with their licenses, and regularly updated via npm audit or equivalent tools.

Library / Tool	Version (at dev time)	Purpose	License	Security Notes / CVEs
React	18.x	Frontend UI framework	MIT	Actively maintained. Low CVE risk if kept updated.
Tailwind CSS	3.x	Utility-first CSS styling	MIT	No known active CVEs.
Node.js	18.x LTS	Server runtime	MIT	Node should be patched regularly; CVEs depend on underlying modules.
Express.js	4.x	Backend API routing	MIT	Past CVEs mitigated in latest versions. Avoid deprecated middleware.
MongoDB (Atlas)	N/A (Managed service)	Cloud NoSQL Database	SSPL / Server-Side Public License	Managed securely by MongoDB Atlas. Admin roles restricted.

Library / Tool	Version (at dev time)	Purpose	License	Security Notes / CVEs
jsonwebtoken	9.x	Auth token management (JWT)	MIT	Ensure secure secret handling and expiration setup. CVEs on <8.5.1.
bcrypt	5.x	Password hashing	MIT	Uses native bindings; avoid downgrading for security.
dotenv	16.x	Environment config management	BSD-2-Clause	No known risks, avoid exposing .env in version control.
cors	2.x	Cross-origin request handling	MIT	Safe when configured explicitly (not '*').
mongoose	7.x	MongoDB ODM	MIT	Keep updated to avoid query injection bugs.
OpenAI SDK	4.x	AI feedback integration	MIT	Subject to OpenAI's API terms; no CVEs, but check rate/security limits.
Google OAuth2	API-based	User authentication	Google APIs Terms	Tokens are exchanged via HTTPS; use scopes and refresh tokens properly.
Vercel	N/A	Frontend hosting	Proprietary	Free tier usage; secure via dashboard config.
Railway	N/A	Backend hosting	Proprietary	Free tier usage; rate limit and cold start risk.

4. Installation & Maintenance Instructions

4.1 System Requirements

To install and run the HELP Web App locally or in a development environment, the following minimum system requirements are recommended:

Requirement	Specification
Operating System	Windows 10+, macOS 11+, or Ubuntu 20.04+
Processor	2.0 GHz dual-core or higher
RAM	4 GB minimum (8 GB recommended for smooth performance)
Disk Space	500 MB for project files + MongoDB storage
Node.js Version	v18.x or later
MongoDB	MongoDB Atlas (cloud) or MongoDB Community Edition v6+
Web Browser	Chrome, Firefox, Safari, or Edge (latest versions)
PDF Viewer	For downloading and viewing feedback reports

4.2 Installation Instructions

Backend (Express + MongoDB)

1. Clone the GitHub repository:

```
git clone https://github.com/Tareq0377/GQ.git
cd GQ/backend
```

2. Install backend dependencies:
3. Set up environment variables in .env:
4. Start the backend server:

Frontend (React + Vite)

1. Navigate to the frontend folder:
2. Install frontend dependencies:
3. Start the development server:

The app should now be running on <http://localhost:5173> (frontend) and <http://localhost:5000> (API).

4.3 Updating & Patching

To update the project:

- Pull the latest changes from GitHub:
- Reinstall dependencies if the package.json has changed:
- Restart the servers to apply updates:

To patch backend logic or scoring:

- Modify the logic in `/controllers/quizController.js`
- Rebuild or restart the server to apply patches

4.4 Dependency, CVE & License References

Each dependency is installed via npm and listed in package.json. For vulnerability and license checks, run:

```
npm audit
```

```
npm list --depth=0
```

4.5 Extending the System

To extend the HELP Web App:

- **Add a new quiz question:**
 - o Go to Admin Panel → Manage Questions → Add Question
 - o Assign a domain/category and question type
- **Add a new admin or researcher:**
 - o Go to Admin Panel → Manage Admins → Add Admin
- **Modify scoring logic:**
 - o Update `getScoreByDomain()` in frontend
 - o Adjust logic in `quizController.js` if needed
- **Add analytics or AI feedback:**
 - o Modify `AdminAnalytics.jsx` for new charts
 - o Extend `scoreInterpretations` DB collection with new domains

- **Modify Benchmark Values:**
 - o Go to Admin Panel → Scoring Criteria → Select Domain
 - o Modify selected domain with new values.

4.6 API Keys & Environment Configuration

The HELP Web App relies on multiple external APIs and services. These credentials are stored in an .env file located at the root of the backend project (/server/.env). The current environment variables were created under the **Good Question team account** for development purposes.

Required Environment Variables

Variable	Description
MONGODB_URI	MongoDB Atlas connection string
JWT_SECRET	Secret key for signing user authentication tokens
GOOGLE_CLIENT_ID	Google OAuth2 Client ID for login
GOOGLE_CLIENT_SECRET	Google OAuth2 Client Secret
OPENAI_API_KEY	API Key for accessing GPT-based feedback

Important Notes

- **These credentials are not valid for production use.** Future deployments must **register their own** accounts and replace the keys in the .env file.
- The .env file is **excluded from version control** via .gitignore and must be manually added in each deployment.
- Sample environment structure is provided in /server/.env.example.

4.7 Resources:

4.7.1 Learning Resources

- [React Docs](#)
- [Express Docs](#)
- [MongoDB Docs](#)
- [Vite Docs](#)
- [Framer Motion](#)
- [Tailwind CSS](#)

4.7.2 Domain Knowledge Resources

The HELP Web App is based on the concept of **Death Literacy**, which refers to a person's knowledge, skills, and confidence to engage with end-of-life matters, including death, dying,

and bereavement. To ensure the platform is evidence-based and aligned with current practices, the team consulted several authoritative sources on community-centered palliative care, aged care planning, and health literacy.

These resources are especially useful for future developers, researchers, or health professionals seeking to understand the context behind the quiz structure, the SEAK scoring model (Skills, Experience, Action, Knowledge), and the design of personalized feedback.

Recommended Resources:

- **The Groundswell Project – Death Literacy Index**
<https://www.thegroundswellproject.com/death-literacy-index>
- **La Trobe University – End of Life Directions for Aged Care (ELDAC)**
<https://www.eldac.com.au/>
- **Palliative Care Australia – Resources and Policy**
<https://palliativecare.org.au/>
- **WHO – Palliative Care Global Overview**
<https://www.who.int/news-room/fact-sheets/detail/palliative-care>
- **YouTube – Death Literacy & Community Empowerment**
https://www.youtube.com/watch?v=KMbF7_HZtiQ

These materials informed both the quiz content and the AI-driven feedback logic. They also provide a foundation for further development, including future training, community education, and research extensions.

5. User Stories and Testing

5.1 User Story Definitions

Each user story is framed in the format:

As a [user role], I want to [action], so that I can [goal].

ID	User Story
U-001	As a public user, I want to view the landing page, so that I can understand what the app is about.
U-002	As a user, I want to take the quiz, so that I can assess my death literacy.
U-003	As a user, I want to receive feedback after the quiz, so that I understand my strengths and weaknesses.
U-004	As a user, I want to download my feedback report, so that I can reflect on it later.
U-005	As a user, I want to resume the quiz, so that I can complete it at my own pace.
U-006	As an admin, I want to view all user submissions, so that I can analyze responses.
U-007	As an admin, I want to export quiz results, so that I can process data externally.
U-008	As an admin, I want to add/edit/delete questions, so that the quiz content stays relevant.
U-009	As an admin, I want to manage admin accounts, so that access remains secure.
U-010	As an admin/researcher, I want to view analytics, so that I can understand community trends.

5.2 User Story Dictionary

U-001

As a public user, I want to view the landing page, so that I can understand what the app is about.

Description: This story involves unauthenticated users accessing the HELP Web App's landing page. It provides a general overview of the platform, its purpose, and the importance of death literacy. It also includes links to resources and navigation guidance for signing up or logging in.

U-002

As a user, I want to take the quiz, so that I can assess my death literacy.

Description: This story describes the core functionality of the HELP platform. Authenticated users can begin the interactive quiz, which includes a series of Likert, multiple-choice, and conditional questions designed to assess their knowledge and comfort around end-of-life topics.

U-003

As a user, I want to receive feedback after the quiz, so that I understand my strengths and weaknesses.

Description: After quiz completion, users receive immediate feedback based on their scores in the SEAK domains (Support, Experience, Action, Knowledge). The feedback helps them reflect on their capabilities and areas for growth in end-of-life planning.

U-004

As a user, I want to download my feedback report, so that I can reflect on it later.

Description: Users are given the option to download a PDF version of their quiz results. This report includes color-coded scores, domain-specific feedback, and AI-generated suggestions that users can revisit anytime.

U-005

As a user, I want to resume the quiz, so that I can complete it at my own pace.

Description: Users who leave the quiz midway can return later and resume from where they left off. This supports flexible engagement, especially for users who need more time to reflect on questions.

U-006

As an admin, I want to view all user submissions, so that I can analyze responses.

Description: Admins have access to all completed quiz submissions. They can view individual user responses, scores, and timestamps to better understand engagement levels and support needs.

U-007

As an admin, I want to export quiz results, so that I can process data externally.

Description: Admins can filter submissions and export them in CSV format for deeper analysis, reporting, or research purposes. This supports data portability and institutional reporting needs.

U-008

As an admin, I want to add/edit/delete questions, so that the quiz content stays relevant.

Description: Through the Admin Panel, administrators can manage the quiz question bank. They can update text, type, options, domain category, and conditional logic, ensuring the quiz remains current and accurate.

U-009

As an admin, I want to manage admin accounts, so that access remains secure.

Description: Admins can add, edit, or delete admin/researcher accounts and control access roles. This supports secure delegation and protects sensitive data from unauthorized users.

U-010

As an admin/researcher, I want to view analytics, so that I can understand community trends.

Description: The analytics dashboard provides visualizations such as domain score distributions, regional insights, and participation trends. This helps stakeholders monitor platform impact and guide strategic decisions.

5.3 User Story Testing Table

Story ID	Test Case	Expected Outcome	Status
U-001	Navigate to landing page as guest	User sees About Us, Quiz, Resources, and Contact Us	Pass
U-002	Start quiz as logged-in user	Quiz loads and progresses correctly with visual tracker	Pass
U-003	Submit quiz and view results	Scores and domain-based feedback are displayed	Pass
U-004	Click "Download Report" after submission	PDF downloads with accurate info	Pass
U-005	Refresh page mid-quiz and return later	User is prompted to resume where they left off	Pass
U-006	Admin logs in and opens "Submissions" tab	List of submissions is displayed with filters	Pass
U-007	Click "Export CSV" on submissions tab	Submissions CSV downloads successfully	Pass
U-008	Add a new question from admin panel	New question appears in quiz dynamically	Pass
U-009	Edit/delete researcher account	Changes reflect in admin list instantly	Pass
U-010	Open analytics and select region/domain filters	Charts update dynamically with correct stats	Pass

5.4 Functional Testing Summary

- All components tested via **manual UI testing**
- Key features tested across **Chrome, Firefox, Edge**
- LocalStorage used for quiz resume functionality
- Backend tested using **Postman** and sample payloads

6. Software Architecture & Diagrams

This section outlines the system's architectural design using standard software diagrams. It includes the high-level architecture, sequence diagrams, and use case diagrams to illustrate the structure and flow of the HELP Web App.

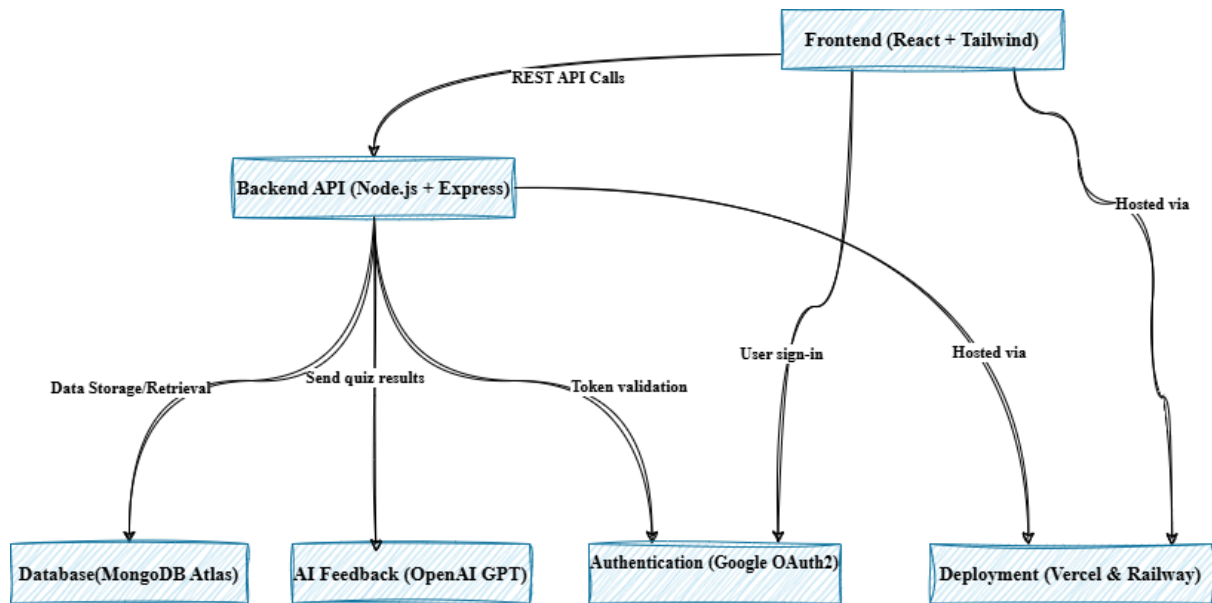
6.1 High-Level Architecture Diagram

The HELP Web App follows a modular MERN stack architecture, designed for scalability and future deployment flexibility:

[Client (Browser)] ↔ [React (Vite + Tailwind CSS)] ↔ [Express API (Node.js)] ↔
[MongoDB Atlas (Cloud Database)]

System Component Breakdown:

- **Frontend:**
Built using React with TailwindCSS, bundled via Vite for fast local development and responsive design. It handles the quiz UI, conditional logic, and user navigation.
- **Backend:**
Powered by Node.js and Express, the backend manages:
 - API routing and validation
 - Scoring and SEAK domain logic
 - User authentication (including Google OAuth2)
 - AI-powered feedback integration via OpenAI API
- **Database:**
Uses MongoDB Atlas, a secure cloud-hosted NoSQL database. It stores:
 - User profiles and roles
 - Quiz questions, categories, and conditional rules
 - Submission results and score breakdowns
 - Feedback interpretation text by domain
- **Deployment:**
As of now, the application is actively hosted on GitHub and run in a local development environment.
Final deployment (e.g., Vercel for frontend, Railway or Render for backend) is pending client decision on whether to integrate with an existing platform or proceed with a standalone deployment.



System Architecture Block Diagram

6.2 Sequence Diagram

This diagram shows the quiz flow from a user logging in to receiving results:

Actors: User → React App → Express Server → MongoDB

Sample Sequence:

1. User logs in (frontend → backend → DB for auth)
2. Starts quiz → Fetches dynamic questions from MongoDB
3. Submits answers → Backend calculates scores
4. Stores submission in DB → Returns feedback
5. Frontend renders result → Offers download PDF

6.3 Use Case Diagram

The Use Case Diagram illustrates how each user role interacts with the HELP Web App. It highlights the boundaries of functionality, access levels, and role-specific actions, ensuring the system is secure, scalable, and tailored to the needs of different stakeholders.

User Roles and Their Interactions:

- 1) Public User (Unauthenticated Guest)

Users who have not signed in or created an account. Their interaction is limited to public content.

Use Cases:

- View homepage and informational pages (About Us, Resources, Contact Us)
- Submit queries via the Contact Us form
- Register or log in to become an authenticated user

2) Authenticated User (Quiz Taker)

Users who have successfully signed in. Their primary interaction is with the quiz and result generation.

Use Cases:

- Start and complete the structured Death Literacy quiz
- Navigate the quiz with a visual progress bar and question tracker
- Trigger conditional sub-questions based on previous answers
- Pause and resume quiz via localStorage resume logic
- Review and edit answers before final submission
- Receive domain-specific feedback (SEAK: Skills, Experience, Action, Knowledge)
- Download a PDF report with feedback and improvement suggestions
- View past submissions and scores (*future feature*)

3) Admin

Admins have full backend control over content and users. This role is intended for platform maintainers and moderators.

Use Cases:

- Add, edit, or delete quiz questions
- Organize questions by type (Likert, group, MCQ, conditional) and SEAK domain
- Manage admin and researcher accounts
- View all user submissions with filters (date, region, domain)
- Export submission data to CSV for record-keeping
- Access real-time analytics: domain averages, regional trends, score distributions
- Assign submission IDs and review individual answers
- Enable or disable AI feedback features (future setting toggle)

4) Researcher

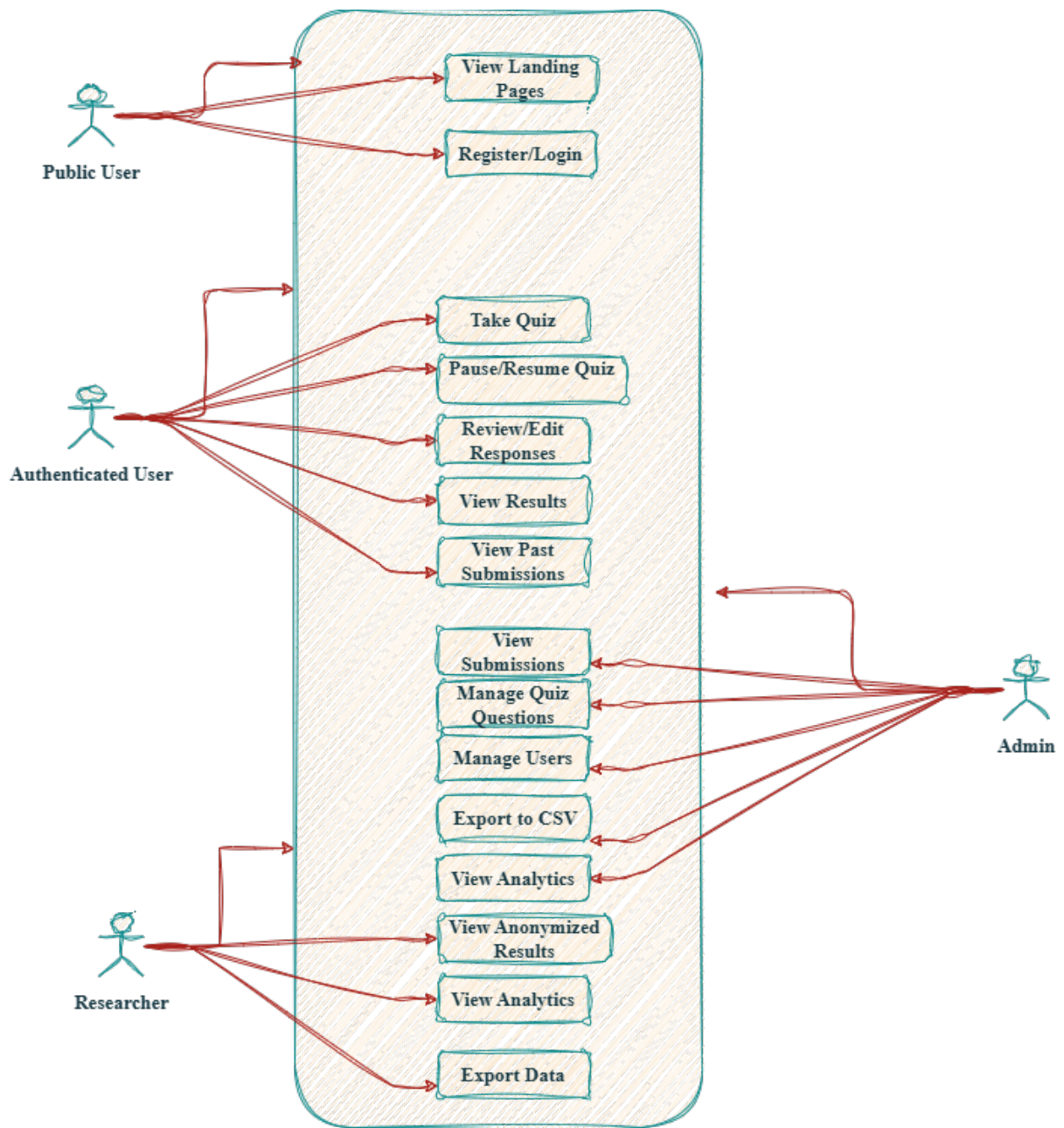
Researchers have read-only access to sensitive data and analytics for academic or community insights. They cannot modify platform content.

Use Cases:

- View anonymized quiz submissions for analysis
- Apply filters to narrow data by region, SEAK domain, or time period
- Export filtered submissions to CSV
- Analyze visual analytics: radar charts, bar graphs, score trends
- View system-wide benchmarks and Death Literacy Index (DLI) scores

Use Case Relationship Summary

Feature	Public User	Authenticated User	Admin	Researcher
<u>View landing pages</u>	✓	✓	✓	✓
<u>Register/Login</u>	✓	=	=	=
<u>Take quiz</u>	✗	✓	=	=
<u>Resume quiz</u>	✗	✓	=	=
<u>Review/Edit answers</u>	✗	✓	=	=
<u>View/download feedback</u>	✗	✓	=	=
<u>Manage questions</u>	✗	✗	✓	✗
<u>Manage user accounts</u>	✗	✗	✓	✗
<u>View/export submissions</u>	✗	✗	✓	✓
<u>View analytics</u>	✗	✗	✓	✓



Use Case Diagram

6.3.1 Cyber Threat Model

The HELP Web App incorporates several safeguards to mitigate common cyber threats across its architecture. Below is an overview of the key threat vectors and the corresponding mitigation strategies implemented:

Threat	Risk Description	Mitigation Strategy
1. Unauthorized Access	Attackers may attempt to bypass authentication to access user or admin content.	All sensitive routes are protected via Google OAuth2 and JWT-based role middleware, restricting access based on user roles (User, Admin, Researcher).
2. Cross-Site Scripting (XSS)	Malicious scripts injected via forms or URLs may compromise user sessions.	The frontend is built with React, which automatically escapes inputs, mitigating most XSS attacks. Inputs are sanitized before being stored.
3. Insecure Communication	Data exchanged between client and server may be intercepted.	The system is designed to operate entirely over HTTPS, ensuring encrypted communication between all components.
4. Data Leakage / Privacy Risks	Personally identifiable information (PII) may be exposed in logs or exports.	Submissions are stored without sensitive personal data; analytics and exports are anonymized by default. No passwords are stored—authentication is delegated to Google.
5. API Abuse / Rate Exploitation	Malicious actors may flood or misuse open APIs.	Backend endpoints are protected with token-based access control, and actions like quiz submission and admin operations are rate-limited and authenticated.
6. Third-party Dependency Vulnerability	Vulnerabilities in external packages (e.g., GPT API, Express) may be exploited.	Dependencies are actively monitored. The project uses only well-maintained open-source libraries and will adopt CVE scanning in future CI/CD pipelines.

This layered threat model supports confidentiality, integrity, and availability (CIA) principles. It ensures the HELP Web App is secure by design while being extensible for future enhancements such as 2FA, audit logging, or vulnerability monitoring tools like Snyk.

6.4 Wireframes

Basic wireframes used during planning:

- Home Page
- Quiz Flow
- Result Page
- Admin Dashboard
- Analytics Tab

A few wireframes as well as hand drawn first drafts are also provided below.

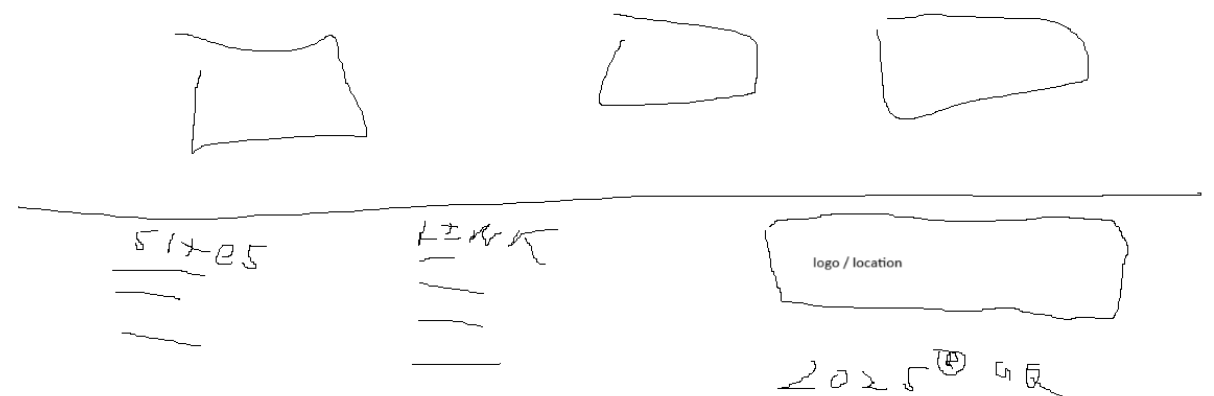
LOGO 1 2 3 4

Jumbotron / image carousel

□ □ □

TAKE QUIZ

Resources



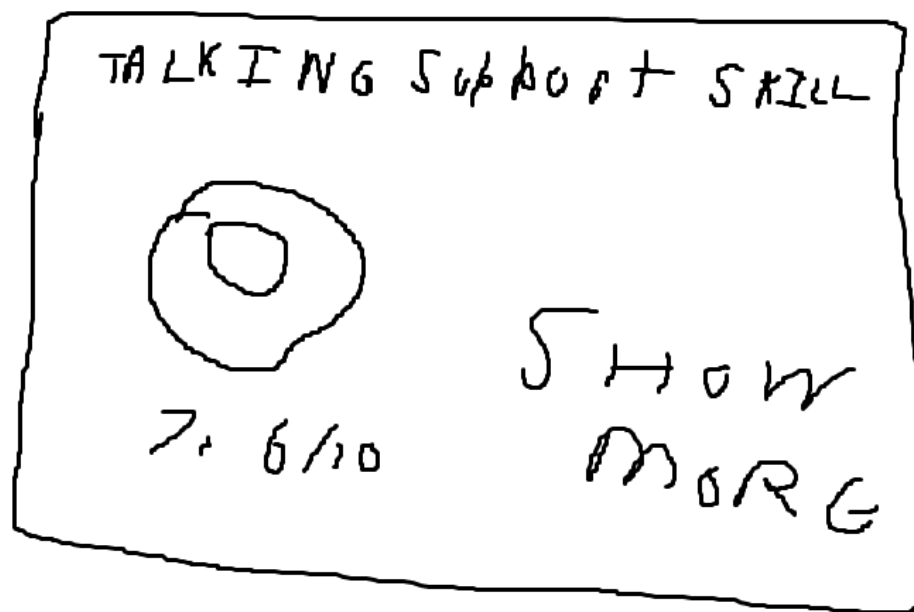
sites

links

logo / location

2025 © GQ

Landing Page First Draft



Domain Score Card First Draft

Final Questionnaire Page with Navigation Bar	Final Personalized Report Page with Navigation Bar
Home About Resources Take the Quiz Contact Us Login / Sign Up	Home About Resources Take the Quiz Contact Us Login / Sign Up
Progress Bar - 70% Completed	Summary of Results
Question Title: How comfortable are you discussing end-of-life topics?	Overall Death Literacy Score: 72%
Likert Scale Options (Strongly Disagree to Strongly Agree)	Key Strengths & Weaknesses
1. Strongly Disagree	Recommendation Categories
2. Disagree	1. Improve Communication
3. Neutral	2. Advance Care Planning
4. Agree	3. Funeral & Memorial Knowledge
5. Strongly Agree	4. Grief & Bereavement Support
Previous (Disabled/Enabled)	Download PDF Button
Next (Active/Disabled)	Explore Resources for Learning
Save & Exit Button	

Wireframe Model of Basic Quiz

6.5 Class Diagram

The class diagram represents the object-oriented structure of the HELP Web App, modeling core system entities, their attributes, and relationships. While implemented in a NoSQL (MongoDB) environment, the object-oriented design aligns with frontend logic and Mongoose schemas.

Classes & Descriptions

1) User

Represents all registered users: general participants, researchers, and admins.

Attributes:

- userId: ObjectId
- name: String
- email: String
- password: String (hashed)
- role: String ('user' | 'admin' | 'researcher')
- region: String
- createdAt: Date

Relationships:

- 1 user → many submissions

2) QuizQuestion

Stores a single question or a grouped question block with optional sub-questions or conditionals.

Attributes:

- questionId: String (e.g., Q16, Q21_2)
- text: String
- type: String (likert, mcq, group, text, etc.)
- options: [String]
- category: String (SEAK domain)
- order: Number
- subQuestions: [QuizQuestion]
- conditionalSub: { triggerValue, subQuestion }
- createdBy: ObjectId

- createdAt: Date

3) Submission

Represents a user's full quiz submission, including responses and scores.

Attributes:

- submissionId: String (e.g., SUB-00123)
- userId: ObjectId
- responses: Object (e.g., { "Q16_1": 4, "Q18_3": 2 })
- scoreBreakdown: Object (e.g., { Skills: 7.3, Action: 6.8 })
- region: String
- createdAt: Date

4) ScoreInterpretation

Maps a benchmark score range to feedback details for each SEAK domain.

Attributes:

- domain: String (e.g., "Experience")
- scoreRange: { min: Number, max: Number }
- label: String (e.g., "Needs Improvement")
- message: String
- recommendation: String

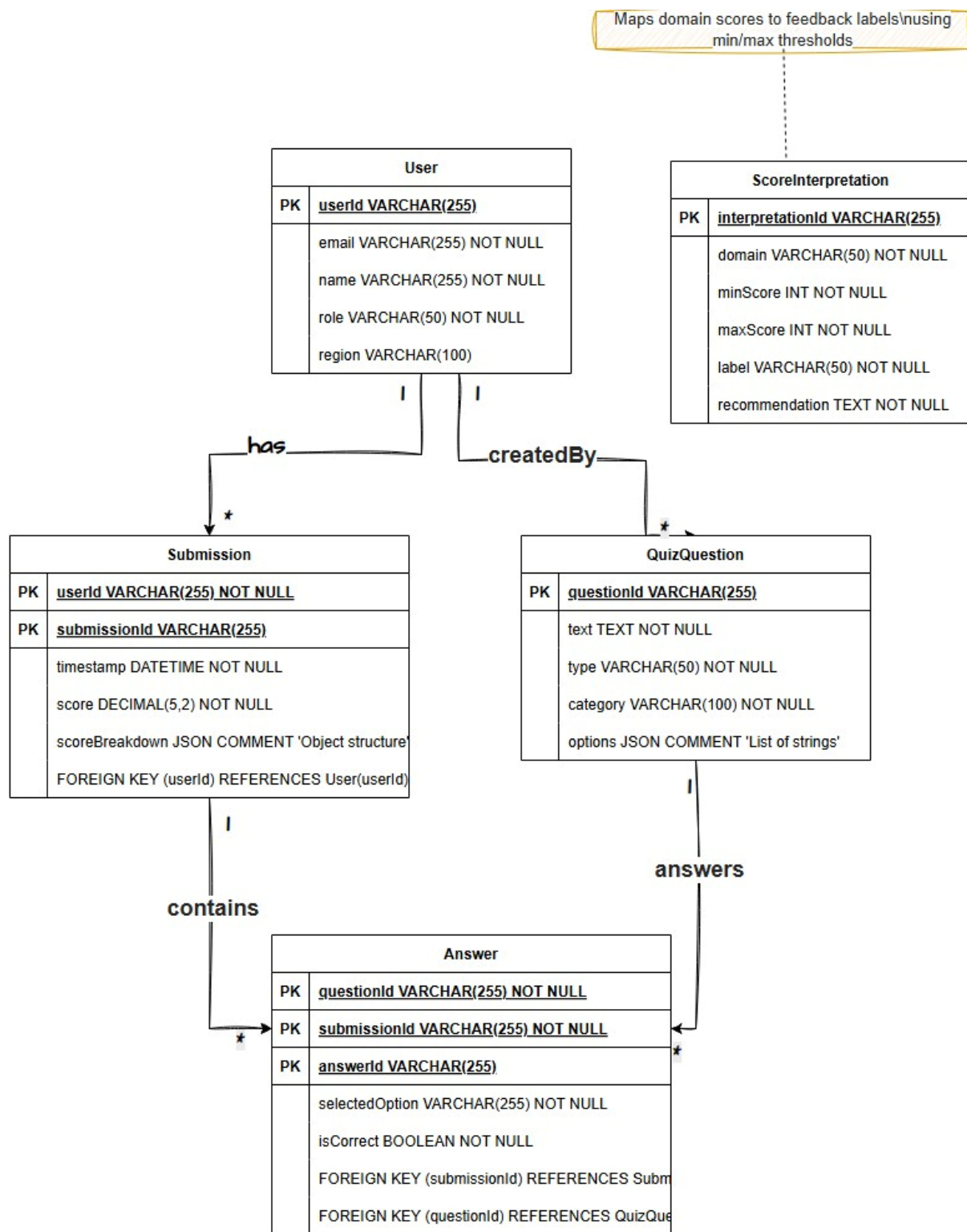
Optional Classes (Future Use)

5) AllInsight

(Optional) Generated feedback object based on submission results.

Attributes:

- submissionId: ObjectId
- insightText: String
- generatedAt: Date
- source: "OpenAI" | "manual"



Class Diagram Drawing

6.6 Entity-Relationship (ER) Diagram

This section describes the **Entity-Relationship model** used to represent the HELP (Healthy End-of-Life Planning) Web App's database structure. While MongoDB is a NoSQL document store and does not enforce foreign key constraints, logical relationships between collections are maintained and mapped as an ER-style schema for clarity.

Core Entities and Relationships:

1) users

Represents all registered individuals, including regular participants, admins, and researchers.

Fields:

- `_id` (ObjectId, Primary Key)
- `name`, `email`, `password` (hashed), `role`, `region`, `createdAt`

Notes:

- Each user may have zero or more quiz submissions.
- `role` determines access (e.g., 'user', 'admin', 'researcher').

2) quizQuestions

Stores the structured question bank including conditional logic and nested sub-questions.

Fields:

- `questionId`, `text`, `type` (likert, mcq, group, etc.)
- `category` (aligned to SEAK domains: Skills, Experience, Action, Knowledge)
- `options`, `order`, `subQuestions`, `conditionalSub`

Notes:

- Not directly linked to submissions but referenced via keys in the responses object.

3) Submissions

Each document represents a completed (or in-progress) quiz by a user.

Fields:

- `_id` (ObjectId)
- `submissionId` (e.g., "SUB-00123")
- `userId` (indirect reference or stored email/name)
- `responses`: Object mapping `questionId` → value (e.g., "Q16_1": 4)

- scoreBreakdown: Calculated SEAK scores per domain
- region, createdAt

Notes:

- One-to-many relationship from users → submissions
- Many-to-many logical mapping to quizQuestions via responses

4) scoreInterpretations

Supports dynamic feedback generation based on submission scores.

Fields:

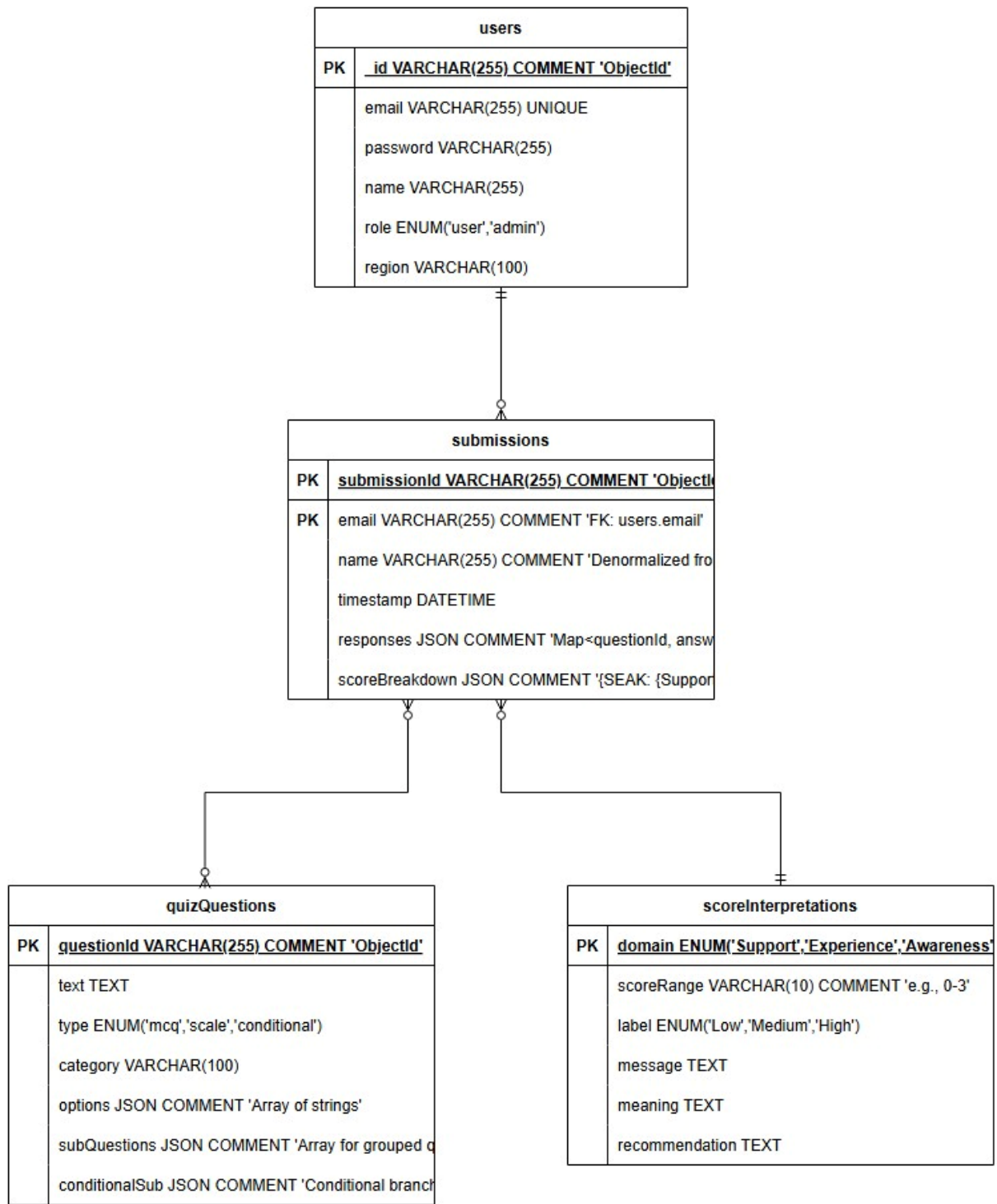
- domain: One of SEAK domains
- scoreRange: { min, max }
- label: e.g., “Needs Improvement”
- message, meaning, recommendation

Notes:

- No direct link to submissions, used at runtime to interpret scoreBreakdown.

Cardinality Summary

From	To	Relationship
User	Submissions	1 — 0..* (One user, many submissions)
Submission	QuizQuestions	* — * (Many-to-many via responses)
Submission	ScoreInterps.	Logical only (referenced during scoring)



Entity – Relationship Diagram

7. Database Design

This section describes the structure of the HELP Web App's MongoDB database, including key collections, fields, and relationships. The database is designed to support a dynamic quiz system, secure user management, and flexible result reporting.

7.1 Database Overview

The HELP Web App uses **MongoDB Atlas** as its database, storing data in document-based collections. All documents are in JSON format, enabling flexible schemas to support dynamic quiz content and conditional logic.

7.2 Entity-Relationship Structure

Although MongoDB is NoSQL and does not use traditional relational schemas, we can represent relationships between collections in an ER-style format. The E-R Diagram was represented in the section 6.6 above.

7.3 Key Collections and Schemas

1) users Collection

Stores information about all registered users (admin, researcher, or regular user).

```
{
  "_id": ObjectId,
  "name": "John Doe",
  "email": "john@example.com",
  "password": "hashed_password",
  "role": "user" | "admin" | "researcher",
  "region": "Victoria",
  "createdAt": ISODate
}
```

2) quizQuestions Collection

Holds all quiz questions, including single questions, grouped questions, and conditional sub-questions.

```
{
  "_id": ObjectId,
  "questionId": "Q16",
```

```

"type": "likert" | "text" | "mcq" | "group" | "conditional",
"text": "I feel confident providing end-of-life care.",
"options": ["Strongly Disagree", "Disagree", ...],
"category": "Skills",
"order": 16,
"subQuestions": [ ... ],
"conditionalSub": {
  "triggerValue": "Australia",
  "subQuestion": { ... }
}
}

```

- 3) quizSubmissions Collection
Captures all submitted responses from users.

```

{
  "_id": ObjectId,
  "userId": ObjectId | null,
  "responses": {
    "Q1": "Australia",
    "Q16_1": 4,
    "Q17_3": 2,
    ...
  },
  "region": "NSW",
  "submissionId": "SUB-00123",
  "scoreBreakdown": {
    "Skills": 7.3,
    "Experience": 6.2,
    ...
  },
  "createdAt": ISODate
}

```

- 4) scoreInterpretations Collection
Maps Benchmark score ranges to custom feedback messages for each domain.

```

{
  "_id": ObjectId,
  "domain": "Skills",
  "scoreRange": { "min": 0, "max": 3.9 },

```

```
"label": "Needs Improvement",  
"message": "You may benefit from learning more about caregiving and  
communication...",  
"recommendation": "Visit our Caregiving Skills & Communication resources."  
}
```

7.4 Additional Notes

- All collections use Mongoose schema validation in the backend.
- Submissions store both flat answers and score summaries.
- The quizQuestions collection is structured to allow admin editing without code changes.
- Conditional logic is handled on both frontend (rendering) and backend (submission validation).

8: Usability Testing

This section describes how the HELP Web App was tested for usability across various user types. The goal was to assess accessibility, ease of navigation, clarity of feedback, and mobile responsiveness, especially considering the intended audience includes older adults.

8.1 Objectives of Testing Scenarios

The usability testing aimed to:

- Identify UI/UX barriers across desktop and mobile devices
- Verify quiz navigation and progress tracking
- Ensure domain-based feedback was understandable and actionable
- Evaluate admin dashboard clarity and analytics readability
- Observe whether older users could complete the quiz independently

8.2 Participants

We conducted informal usability testing with the following user groups:

Participant Type	Count	Notes
University Students	3	General users testing quiz and feedback flow
Older Adults	2	Focus on accessibility and readability
Admin Role Testers	2	Tested admin dashboard, submission export, filters

8.3 Methods

- Participants were asked to use the application without developer help.
- Key tasks were observed:
 - o Start and complete the quiz
 - o Use review/edit feature
 - o View results and download report
 - o For admins: filter submissions, export CSV, view analytics
- Testers were encouraged to speak aloud and flag any confusion or frustration.

8.4 Results of Usability Testing

Positive Feedback

- Quiz flow was intuitive and visually clean
- Users appreciated the progress bar and question tracker
- Domain-based results and PDF download were well received
- Admin panel was described as “straightforward” by users

Identified Issues

Issue	Severity	Resolution
On mobile, Likert buttons occasionally misaligned	Medium	Adjusted layout and spacing via Tailwind CSS
Admin analytics slow to load with >1000 records	Medium	Added CSV export workaround for heavy datasets
Resume quiz caused confusion for some users	Low	Added prompt text to clarify resume behaviour
Older adults found some texts small on mobile	Medium	Increased font size and added spacing globally

8.5 Improvements After Testing

Following usability testing:

- Font sizes and button spacing were optimized for accessibility
- “Resume Quiz” functionality was clarified with instructional text
- Admin export flow was improved with additional filtering options
- Pie and radar charts in analytics were resized and labelled clearly
- New glossary and tooltip features were added to improve clarity

9. Project Management Summary

This section outlines the collaborative development process behind the HELP (Healthy End-of-Life Planning) Web App, covering sprint planning, team contributions, version control practices, communication methods, and project evaluation.

9.1 Project Tracking & Sprints

The team utilized **Jira and sheets** to define epics, track tasks, and manage sprint progress. Each sprint had clear objectives aligned with deliverables:

Sprint	Dates	Key Deliverables
1	Mar 24 – Mar 30	Project setup, GitHub repo, base React & backend init
2	Mar 31 – Apr 6	Quiz flow, Likert logic, resume support, navbar/pages
3	Apr 7 – Apr 13	Feedback logic, result display, PDF export, bug fixing
4	Apr 14 – Apr 20	Backend setup, admin dashboard (basic), MongoDB link
5	Apr 21 – Apr 27	Admin features: edit/delete quiz, view submissions
6	Apr 28 – May 4	Analytics page, SEAK scoring logic, chart integration
7	May 5 – May 11	UI polish, PWA setup, Framer Motion animations
8	May 12 – May 17	Testing, refinements, client feedback, documentation

Screenshots of Jira board and task breakdowns are included in **Appendix A**.

9.2 Team Contributions Overview

Each team member contributed in unique ways. The breakdown below summarizes individual responsibilities:

Name	Role	Key Contributions
Sanidhya Verma	Lead Full-Stack Dev	Quiz logic, resume logic, Admin Panel, Analytics, AI integration, SEAK scoring logic
Manmeet Karwal	Frontend UI	Landing page, carousel, accessibility styling, review flow
Sai Prakash	Backend & Auth	MongoDB setup, token auth, routes, Express APIs

Name	Role	Key Contributions
Md Tariqul Alam	Data & Analytics	Chart generation, PDF export, submission breakdown
Md Faisal Ahmed	Testing & QA	Testing quiz flow, reviewing user feedback, usability testing
Yosef Alshammari	Research & Documentation	User manual, SMD sections, glossary, benchmarks

Sanidhya Verma being the lead handled the most complex logic including conditional question rendering, domain scoring, Benchmark Comparison, and SEAK feedback generation. He also led architectural decisions and sprint management.

9.3 GitHub Version Control

The team maintained a private GitHub repository. Branches were created per sprint/task, and regular commits ensured traceability.

- **Total commits:** ~45+
- **Branches:** master, dev, feature/frontendUI, feature/frontend-polish, feature/backendSetup, feature/newQuizTesing, feature/final-polishing, feature/quiz-logic, etc.
- Pull requests were reviewed and merged weekly.
- Screenshots of commit history are available in **Appendix B**.

9.4 Team Communication

Teams, Whatsapp was used for real-time discussions, issue clarifications, and daily check-ins. We held:

- **Weekly planning meetings**
- **Mid-week progress updates**
- **Final week retrospectives**

Evidence of communication is included in **Appendix C**, demonstrating shared planning, role clarifications, and problem-solving exchanges.

9.5 Evaluation & Lessons Learned

The project progressed steadily through well-structured sprints. Initial delays in backend setup were offset by parallel frontend development. Admin panel complexity and scoring logic required multiple rewrites due to evolving client requirements.

Clear role-based routing, scalable architecture, and real-time feedback generation were key achievements. Team communication and documentation improved with each sprint.

Future improvements include earlier CI/CD integration, broader test coverage, and improved sprint velocity tracking via Jira.

10. Glossary, Indexes, and Final Note

10.1 Glossary of Terms

Term	Definition
Death Literacy	A person's knowledge and confidence in managing end-of-life conversations and care.
SEAK	An acronym for Skills, Experience, Action, Knowledge — four domains used in scoring.
Likert Scale	A seven-point or five-point scale from “Strongly Disagree” to “Strongly Agree” used to measure attitudes.
Grouped Question	A question with multiple related question which share same heading.
Conditional Question	A sub-question that appears only if a specific response is given (e.g., if country = Australia).
Admin/Researcher	Users with elevated permissions to manage content, view submissions, and access analytics.
CSV Export	A downloadable spreadsheet format of user submissions or results.
MongoDB Atlas	Cloud-hosted NoSQL database used to store all user, quiz, and result data.
React	Frontend JavaScript framework used to build the user interface.
Express	Node.js web application framework for backend APIs.
JWT	JSON Web Token, used to manage secure user authentication.
Tailwind CSS	A utility-first CSS framework for styling the frontend.
Framer Motion	An animation library used for transitions and interactive elements.

10.2 Indexes

Tools & Technologies

React – 3, 8

Node.js – 3, 4, 8

MongoDB – 3, 4, 7

JWT – 4, 7

Tailwind CSS – 3, 4

Express – 3, 4

CSV Export – 5, 6

Radar Chart – 6
Likert Scale – 4, 5

Key Concepts

Death Literacy – 1, 4, 5
Scoring – 4, 5, 7
Feedback – 5, 6
Conditional Logic – 4, 7
Admin Panel – 5, 6, 7

10.3 Final Notes

The HELP Web App project was developed with accessibility, clarity, and community impact in mind. Through iterative design, collaborative development, and user feedback, the platform is now well-positioned to support research, public awareness, and resource guidance in the domain of end-of-life planning.

We recommend future developers consider:

- Implementing multi-language support
- Adding audio/text-to-speech options for accessibility
- Enabling custom report generation per organization
- Expanding AI-powered insights using OpenAI or similar APIs

Special thanks to the teaching team, client representatives, and all testers for their valuable input throughout the development process.

10.5 Appendix

This Section will have all the Images for references. Please check below for subsections

10.5.1 Appendix A

This Section holds layout of the Jira stories and tasks to be used as a sample and guide for further development.

	GQ-40	Implement quiz logic for scoring and results	
	GQ-42	Create a function to store quiz scores for each user	
	GQ-41	Implement logic to calculate scores based on selected answers	
	GQ-45	Test the scoring logic and feedback system with multiple question types	
	GQ-43	Implement a feedback system to display the correct answer after each question	
	GQ-44	Add a progress bar showing how far along the quiz the user is	
	GQ-46	Generate personalized reports based on answers	
	GQ-47	Create a report template to display results in a user-friendly format (e.g., score, areas of improvement)	
	GQ-48	Implement logic to map answers to specific feedback based on score ranges	
	GQ-49	Add a "Download Report" feature to export results as a PDF	
	GQ-50	Design a user interface for displaying personalized reports, including graphical feedback (charts, pro...	
	GQ-51	Conduct testing on personalized report generation with various data sets	

Jira tasks

GQ-24	As a user, I want the app layout to be responsive, so that I can us...	 Story	QUIZ UI AND USER ...	DONE	SV	8
GQ-25	Implement a responsive grid layout using Tailwind CSS	 Task	QUIZ UI AND USER ...	DONE	MK	5
GQ-26	Create a navigation bar with active state indication	 Task	QUIZ UI AND USER ...	DONE	MK	5
GQ-27	Implement a sticky footer with essential links	 Task	QUIZ UI AND USER ...	DONE	MK	5
GQ-28	Ensure responsiveness across different screen sizes	 Task	QUIZ UI AND USER ...	DONE	SV	4
GQ-29	Perform browser compatibility testing and fix layout issues if any	 Task	QUIZ UI AND USER ...	DONE	SV	4
GQ-30	As a user, I want to see quiz questions and answer options clearl...	 Story	QUIZ UI AND USER ...	DONE	SR	8
GQ-31	Create a dynamic question component	 Task	QUIZ UI AND USER ...	DONE	MK	5
GQ-32	Implement answer selection with visual feedback	 Task	QUIZ UI AND USER ...	DONE	SV	5
GQ-33	Add a progress indicator	 Task	QUIZ UI AND USER ...	DONE	MA	3
GQ-34	Ensure accessibility (Keyboard navigation and screen reading su...	 Task	QUIZ UI AND USER ...	DONE	YA	3
GQ-35	Conduct internal UI testing and fix usability issues	 Task	QUIZ UI AND USER ...	DONE	MA	4
GQ-36	As a team, we need to present our work at the online trade show...	 Story	GATHER TOWN TRA...	DONE	MK	5
GQ-37	Attend the morning session on 2nd April, Wednesday	 Task	GATHER TOWN TRA...	DONE	SV	4

Jira Stories with different epics

Key:	Summary:	Issue type:	Epic:	Status:	Assignee:	Story points
QQ-40	Implement quiz logic for scoring and results	Story	QUIZ LOGIC AND REPORT G...	DONE	SV	15
QQ-50	Design a user interface for displaying personalized reports, including graphical feedba...	Task	QUIZ LOGIC AND REPORT G...	DONE	SR	4
QQ-51	Conduct testing on personalized report generation with various data sets	Task	QUIZ LOGIC AND REPORT G...	DONE	MA	2
QQ-49	Add a "Download Report" feature to export results as a PDF	Task	QUIZ LOGIC AND REPORT G...	DONE	YA	3
QQ-47	Create a report template to display results in a user-friendly format (e.g., scores, answers...	Task	QUIZ LOGIC AND REPORT G...	DONE	MK	4
QQ-48	Implement logic to map answers to specific feedback based on score ranges	Task	QUIZ LOGIC AND REPORT G...	DONE	MK	5
QQ-45	Test the scoring logic and feedback system with multiple question types	Task	QUIZ LOGIC AND REPORT G...	DONE	MA	2
QQ-46	Generate personalized reports based on answers	Story	QUIZ LOGIC AND REPORT G...	DONE	MK	13
QQ-43	Implement a feedback system to display the correct answer after each question	Task	QUIZ LOGIC AND REPORT G...	DONE	MK	3
QQ-44	Add a progress bar showing how far along the quiz the user is	Task	QUIZ LOGIC AND REPORT G...	DONE	YA	2
QQ-41	Implement logic to calculate scores based on selected answers	Task	QUIZ LOGIC AND REPORT G...	DONE	SV	5
QQ-42	Create a function to store quiz scores for each user	Task	QUIZ LOGIC AND REPORT G...	DONE	SR	3

Jira tasks allotted to members along with story points

10.5.2 Appendix B

This Section holds the commit history of the app on GitHub as well as the contribution chart which can be used as a sample and guide for further development.



Contribution chart

Commits		
🔍 dev	👤 All users	📅 All time
Commits on Jun 6, 2025		
Merge pull request #16 from Tareq0377/master	Verified	c5568c9
Merge pull request #15 from Tareq0377/feature/final-polishing	Verified	ac7e20b
Merge pull request #14 from Tareq0377/feature/final-polishing	Verified	53881d2
Final Push with all enhancements under the scope		ec14bd3
Merge pull request #13 from Tareq0377/feature/final-polishing	Verified	e74301b
Commits on Jun 4, 2025		
Resources page updates with domain mapping		b39593d
toast message Implementation on quiz/ login page		f2a6879
Final Sidebar fixes		

Commit History

Commits on May 29, 2025		
Merge pull request #12 from Tareq0377/master	Verified	01e192b
Merge pull request #11 from Tareq0377/dev	Verified	15600a7
Fixed errors on signup and signin page		fa47c86
Partial: Toast notification enabled instead of inline notification.		014c6de
Improved User Profile Section and Tweaks to save changes after code deletion		d1e06c8
Commits on May 28, 2025		
Improved RBAC for Researchers		f30ec90
Commits on May 27, 2025		
Refined various components and animation for final touch ups		b147e12
Commits on May 26, 2025		
Updated Analytics Page with new graphs, still needs polishing		hbae48a

Commit History

Revamped Dashboard with a few insights		35b2585
Final Scoring Logic Implementation with Updated Dashboard		a0c09c0
Commits on May 25, 2025		
Fixed the country dropdown as well as the conditional sub question logic.		090b6a8
Admin Manage Question now supports multiple question and sub question editing functions		8f55bb3
Commits on May 24, 2025		
Fixed issue with region not showing up on result page as well as pdf report		244d954
Fixed Question and subquestion label on review page		a34a8f4
Fixed the green and grey progress number for quiz		9d430b9
Big Update from remote machines of all team members		cdc4002
Commits on Apr 29, 2025		
Revised authentication routes, fixed user login errors and quiz flow bugs		e43ad60

Commit History

Implemented api calls for backend quiz and solved the user/admin rbac	e18135c	<>
Backend setup and init	b96cd4b	<>
Final Ui Push before backend	c188a7b	<>
Commits on Apr 14, 2025		
Quiz wokring with Json files and complete overhaul	7a95f81	<>
Commits on Apr 13, 2025		
testing the new json file for quiz	c2e28f7	<>
Finalized quiz frontend features like logo and main image before JSON data integration	d14ab04	<>
Commits on Apr 12, 2025		
Signup Page and Login Page Updates	a1bbc14	<>
Commits on Apr 11, 2025		
Updated Readme.md	7a640bf	<>

Commit History

Update README	e168048	<>
Commits on Apr 8, 2025		
Complete quiz logic with result page, scoring, and PDF export	39a08c1	<>
Merge pull request #2 from Tareq0377/feature/frontendUI	Verified cdf93be	<>
add few folders to client side	Verified fb889d4	<>
Initial Frontend Skeleton	23b66bb	<>
Commits on Apr 6, 2025		
update of project structure	e8543ee	<>
Commits on Apr 5, 2025		
initialize project folder structure	d1e039c	<>
Initial commit	Verified f817ceb	<>

Commit History

10.5.3 Appendix C

This Section holds the communication methods and protocols which can be used as a sample and guide for further development.

Good Question

ChatShared

Meet now6

YOSEF ALSHAMMARI01-04 22:00

YA

<https://latrobe.zoom.us/j/9553482038?pwd=dRabAilA1nwOd0gbf8lUUmXnzu9wmg.1>

01-04 22:28

Meeting ended 48m 40s

01-04 23:07

SAI PRAKASH RAVICHANDRAN started a meeting.

01-04 23:07

SAI PRAKASH RAVICHANDRAN started recording to the cloud

01-04 23:09

SAI PRAKASH RAVICHANDRAN stopped recording.

01-04 23:10

Recording has been saved to SAI PRAKASH RAVICHANDRAN's OneDrive

01-04 23:10

Meeting ended 3m 6s

Good Question

01 April 2025 23:07 - 23:09

View recap

Content

Transcript

images.jpeg

2m 22s

03 April

03-04 18:42

SAI PRAKASH RAVICHANDRAN started a meeting.

03-04 18:42

Meeting ended 26s

26 May

26-05 19:28

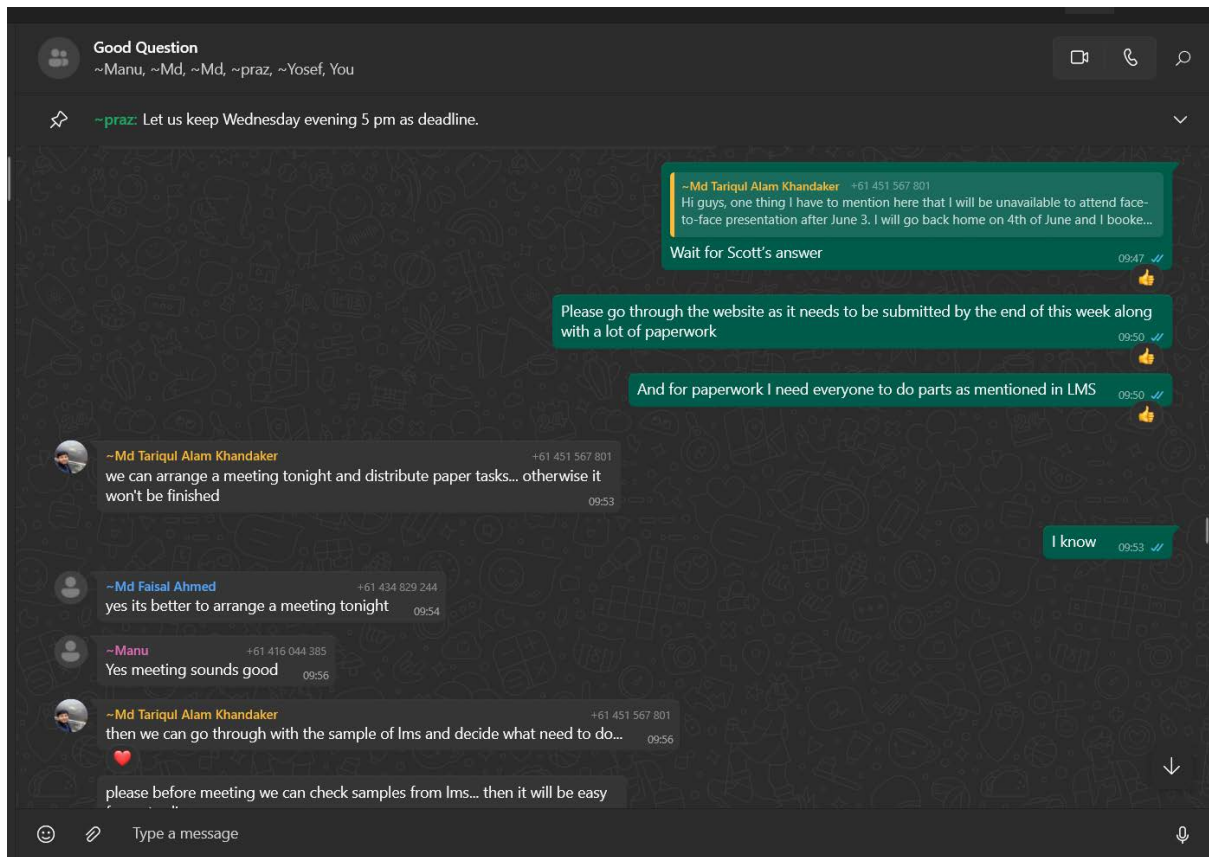
YOSEF ALSHAMMARI started a meeting.

26-05 19:28

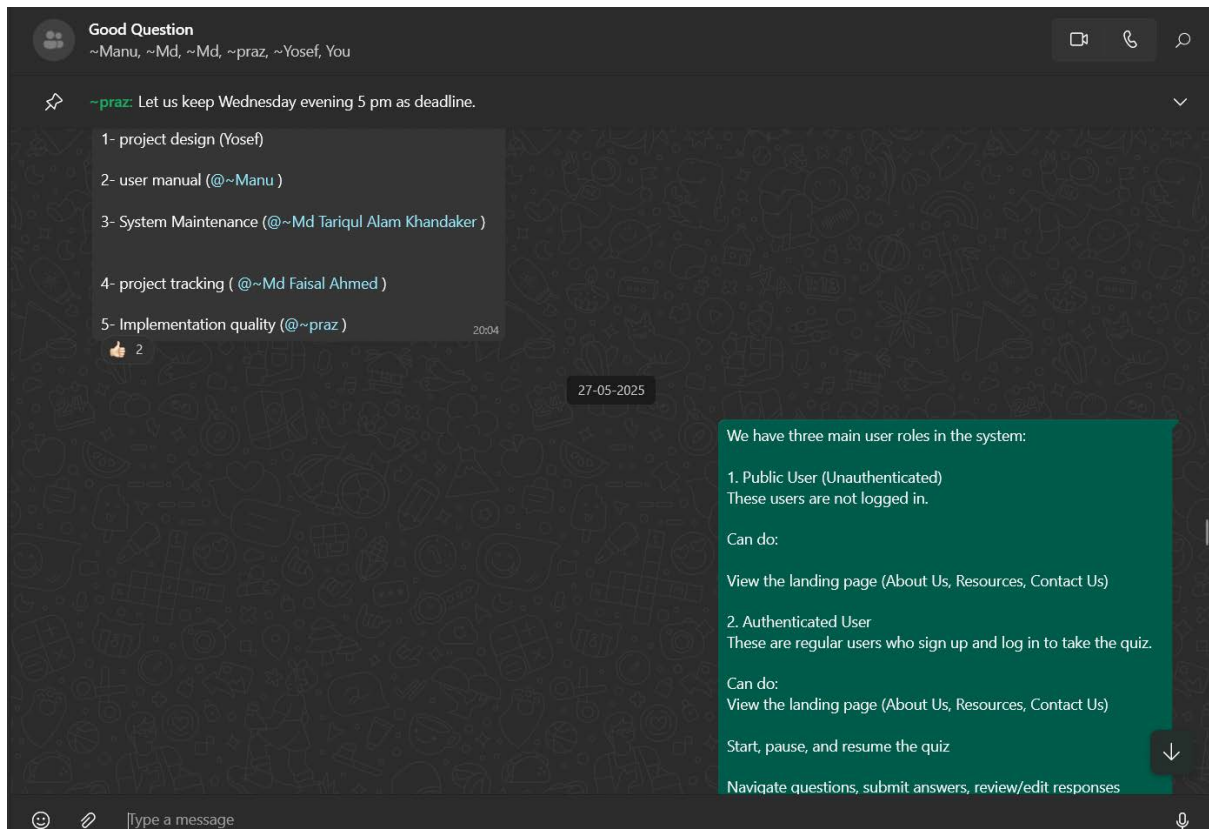
Meeting ended 7s

Type a message

Recording our meeting so that everyone is updated



Assessing and communicating



Explaining concepts to team